82
Pass

Santosh Kumar Sahoo

PDF generated at: 17 Feb 2026 09:25:31 UTC

View this report on HackerRank

Score

82% • 143.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 43 min 21 sec on 17 Feb 2026 11:22:19 IST

Candidate Information

Email	santoshk.dev97@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	17 Feb 2026 11:22:19 IST
Time taken	43 min 21 sec/ 90 min
Work Experience	4 years
Invited by	Kalyani
Invited on	16 Feb 2026 21:06:56 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	50%

2	Spring Boot Intermediate	83%
3	Problem Solving Intermediate	68%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	70%	Hard	50%
TDD	0%	Spring	50%
JPA	50%	Spring Data	50%
Transactional	50%	Medium	83%
Sorting	84%	Greedy	84%
Arrays	68%	Two Pointers	68%
Java	100%		

Questions

Spring Boot (Advanced) • 17.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 26 sec	✓

2	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 40 sec	✗
3	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	4 min 1 sec	✓
4	Security Custom Login Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 50 sec	✓
5	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	57 sec	✓

Problem Solving (Intermediate) • 51 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Capable Models Coding	Problem Solving (Intermediate)	51/75	-	22 min 5 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: DataVille's Optimal Storage Coding	Java (Intermediate)	75/75	-	10 min 35 sec	✓

1. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

2. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
   {
      logWriter.write("Entered Test 1");
      assertTrue(someClassToTest.someMethod());
   }

   @Test
   public void test2()
   {
      logWriter.write("Entered Test 2");
      assertTrue(someClassToTest.otherMethod());
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- The output file is not created.
- **The output file is created but not deleted.**
- The output file is created and eventually deleted.
- The create and delete blocks will not be executed.

No comments.

3. Spring Data Transactional

Partially correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
    @Id
    private Long id;
    private String name;
    private Integer courseLength;
    private Integer courseLevel;
    @JsonBackReference
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id")
    private Student student;
```

```
    private boolean inProgress;
    //getter & setters
}
```

Student.java

```
@Entity
public class Student {
    @Id
    private Long id;
    private String name;
    private String surname;
    @JsonManagedReference
    @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
    private List<Lesson> lessons;
    //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
```

```
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] It will try to delete the student then the DataIntegrityViolationException will be thrown.
- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".
- [x] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.
- [ ] It will throw a RuntimeException with the message "Student not found".
- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

4. Security Custom Login

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which of the following options can be used to create a custom login page in Spring Security?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .and() .httpBasic(); }
- ○ protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login"); }
- ● **protected void configure(HttpSecurity http) throws Exception { http .authorizeRequests() .anyRequest().authenticated() .and() .formLogin() .loginPage("/login") .permitAll(); }** ✓
- ○ None of the above

No comments.

---

5. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Configuration class annotated with @EnableGlobalMethodSecurity
- ○ Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)
- ● **Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)** ✓
- ○ None above

No comments.

6. Capable Models

Partially correct

Coding | Medium | Sorting | Greedy | Arrays | Two Pointers

Question description

Given *n* machine learning models, each with an associated cost and feature compatibility:

- *cost[i]* represents the cost of the ith model
- *featureAvailability[i]* is a binary string indicating suitability for two distinct features:
  - "00": not equipped for either feature
  - "01": suitable for feature A but not feature B
  - "10": suitable for feature B but not feature A
  - "11": suitable for both features

A set of models is *k*-capable if the number of models suitable for feature A and the number suitable for feature B are both greater than or equal to *k*.

For each value of *k* from 1 to *n*, determine the minimum cost required to assemble a *k*-capable set of models. Return an array of *n* integers, where the ith integer represents the minimum cost for an *i*-capable set. If no *i*-capable set exists, the ith integer should be -1.

Example

n = 6
cost = [3, 6, 9, 1, 2, 5]
featureAvailability = ["10", "01", "11", "01", "11", "10"]
featureAvailability = [“10”, “01”, “11”, “01”, “11”, “10”]

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost
1	[5]	[5]	[5]	2
2	[1, 4, 5]	[1, 5]	[4, 5]	3 + 1 + 2 = 6
3	[1, 3, 4, 5]	[1, 3, 5]	[3, 4, 5]	3 + 9 + 1 + 2 = 15
4	[1, 2, 3, 4, 5, 6]	[1, 3, 5, 6]	[2, 3, 4, 5]	3 + 6 + 9 + 1 + 2 + 5 = 26

For $k \geq 5$, there will be no capable set.
Hence, the answer is [2, 6, 15, 26, -1, -1].

Function Description

Complete the function *getMinimumCost* in the editor with the following parameters:

int cost[n]: the cost of machine learning models

string featureAvailability[n]: the compatibility string of models indicating their suitability for two features

Returns

int[n]: the ith integer is the minimum cost of a set of *i*-capable models

Constraints

- $1 \le n \le 10^5$
- $1 \le cost[i] \le 10^4$
- It is guaranteed that *featureAvailability[i]* is a binary string of length 2.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of machine learning models.
Each of the next *n* lines contains an integer *cost[i]*.
The next line contains an integer, *n*, the number of models.
Each of the next *n* lines contains a string *featureAvailability[i]*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
4      →    cost[] size n = 4
5      →    cost = [5, 6, 10, 1]
6
10
1
4      →    featureAvailability[] size n = 4
10      →    featureAvailability = ["10", "01", "11", "00"]
01
11
00
```

Sample Output

```
10
21
-1
-1
```

Explanation

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost
1	[3]	[3]	[3]	10
2	[1, 2, 3]	[1, 3]	[2, 3]	5 + 6 + 10 = 21

For $k \geq 3$, there is no capable set.

Hence, the answer is [10, 21, -1, -1].

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
2      →    cost[] size n = 2
1      →    cost = [1, 1]
1
2      →    featureAvailability[] size n = 2
10     →    featureAvailability = ["10", "10"]
10
```

Sample Output

```
-1
-1
```

Explanation

There is no possible way of forming a *k-capable* set for all *k* from 1 to *n*.

Interviewer guidelines

▼ SOLUTION

Skills: Sorting, Two Pointers, Greedy Algorithms

To find the minimum cost for models such that the system is k-capable, the ones with "00" are unusable, while "01" and "10" have to be used together always and "11" can be used independently. This is because there should be k models supporting both features A and B.

Also, to find for each k from 1 to n, if the minimum cost of a k-capable system is known, find the next unused model(s) with minimum cost such that it supports both features A and B, and add the cost to find the minimum cost of (k + 1)-capable system.

So, the target is to create an array in increasing order such that each element is the cost of adding model(s) such that it supports both features.

Create three arrays: cost_both containing the models “11”, cost_one containing the models “01”, and cost_two containing the models “10”. Then, sort these arrays in an increasing order. Then, create an array cost_sum such that the ith element in cost_sum is cost_one[i] + cost_two[i], representing the cost to convert one “01” and one “10” into one “11” in sorted order. Then, append the two arrays cost_both and cost_sum and sort them to get the final array of costs.

Optimal Solution:

```
def getMinimumCost(cost, featureAvailability):
    n = len(cost)

    x1 = [x for x, y in zip(cost, featureAvailability) if y == "10"]
    x2 = [x for x, y in zip(cost, featureAvailability) if y == "01"]

    x = sorted(map(sum, zip(sorted(x1), sorted(x2))), reverse=True)
    y = sorted([x for x, y in zip(cost, featureAvailability) if y == "11"], reverse=True)

    ans = []
    px = py = val = 0
    while x or y:
        if not x or (x and y and x[-1] > y[-1]):
            val += y.pop()
        else:
            val += x.pop()
        ans.append(val)
    return ans + [-1] * (n - len(ans))
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n log n)

The sorting algorithm takes O(n log n) which is the highest order of operations performed. Therefore, the time complexity is O(n log n).

Space Complexity - O(n)

There are different auxiliary arrays used to store the costs. Therefore, the space complexity is O(n).

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMinimumCost' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY cost
     *  2. STRING_ARRAY featureAvailability
     */

    public static List<Integer> getMinimumCost(List<Integer> cost, List<String> featureAvailability) {
    // Write your code here
    int n = cost.size();

    List<Integer> both = new ArrayList<>();
    List<Integer> onlyA = new ArrayList<>();
    List<Integer> onlyB = new ArrayList<>();

    for(int i = 0; i < n; i++) {
        String s = featureAvailability.get(i);

        if(s.equals("11")){
            both.add(cost.get(i));
        }
        else if(s.equals("01")){
            onlyA.add(cost.get(i));
        }
        else if(s.equals("10")){
            onlyB.add(cost.get(i));
```

```
        }
    }

      Collections.sort(both);
      Collections.sort(onlyA);
      Collections.sort(onlyB);

      long[] prefixBoth = prefixSum(both);
      long[] prefixOnlyA = prefixSum(onlyA);
      long[] prefixOnlyB = prefixSum(onlyB);

      List<Integer> result = new ArrayList<>();

      for(int k = 1; k <= n; k++) {
        long minCost = Long.MAX_VALUE;

        for(int x = 0; x <= Math.min(k, both.size()); x++) {
            int remaining = k-x;

            if(remaining <= onlyA.size() && remaining <= onlyB.size()){
                long total = 0;

                if(x > 0) total += prefixBoth[x-1];
                if(remaining > 0) {
                    total += prefixOnlyA[remaining-1];
                    total += prefixOnlyB[remaining-1];
                }
                minCost = Math.min(minCost, total);
            }
        }
        if(minCost == Long.MAX_VALUE){
            result.add(-1);
        } else {
            result.add((int)minCost);
        }
      }

    return result;

   }

    public static long[] prefixSum(List<Integer> list) {
        long[] prefix = new long[list.size()];

        long sum = 0;
        for(int i = 0; i < list.size(); i++) {
```

```
            sum += list.get(i);
            prefix[i] = sum;
        }
        return prefix;
    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int costCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> cost = IntStream.range(0, costCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int featureAvailabilityCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<String> featureAvailability = IntStream.range(0, featureAvailabilityCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .collect(toList());

        List<Integer> result = Result.getMinimumCost(cost, featureAvailability);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
```

```
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0648 sec	37.7 KB
TestCase 1	Easy	Sample	Success	1	0.076 sec	37.5 KB
TestCase 2	Easy	Sample	Success	1	0.0609 sec	37.8 KB
TestCase 3	Easy	Hidden	Success	2	0.0538 sec	38.3 KB
TestCase 4	Easy	Hidden	Success	2	0.0618 sec	37.7 KB
TestCase 5	Medium	Hidden	Success	5	0.2624 sec	57.9 KB
TestCase 6	Medium	Hidden	Success	5	0.3446 sec	60.5 KB

TestCase 7	Medium	Hidden	Success	5	0.2832 sec	57.4 KB
TestCase 8	Medium	Hidden	Success	5	0.1951 sec	58.8 KB
TestCase 9	Hard	Hidden	Success	8	0.2155 sec	67.5 KB
TestCase 10	Hard	Hidden	Terminated due to timeout	0	4.0108 sec	74.1 KB
TestCase 11	Hard	Hidden	Terminated due to timeout	0	4.0111 sec	74.7 KB
TestCase 12	Hard	Hidden	Success	8	0.448 sec	72.7 KB
TestCase 13	Hard	Hidden	Terminated due to timeout	0	4.0158 sec	73.8 KB
TestCase 14	Hard	Hidden	Success	8	3.366 sec	87.2 KB

No comments.

7. Java: DataVille's Optimal Storage

Correct

Coding Greedy Sorting Medium Java

Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

Function Description

Complete the function g*etOptimalStorage* in the editor below.

Function Parameters

int databaseStorage[n]: the current organization of the database's storage, with 0s for standard data and 1s for priority data

Returns

int: the minimum number of operations required to minimize transitions between 0s and 1s

Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in \{0, 1\}$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.

Each of the next lines contains an integer, *databaseStorage[i].*

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
0
1
```

Sample Output 0

```
1
```

Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
1         databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]
0
0
1
1
0
0
0
1
```

Sample Output 1

```
2
```

Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;

        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
```

```
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }

        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

Time Complexity - O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

Space Complexity - O(1)

We use only 2 variables, hence the space complexity is O(1).

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
```

```
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int n = databaseStorage.size();

        if(n <= 1) return 0;

        int ones = 0;

        for(int val : databaseStorage) {
            if(val == 1) ones++;
        }

        int zeroes = n-ones;

        if(ones == 0 || zeroes == 0) return 0;

        int swapForOnesFront = 0;
        for(int i = 0; i < ones; i++) {
            if(databaseStorage.get(i) == 0) {
                swapForOnesFront++;
            }
        }

        int swapForZeroesFront = 0;
        for(int i = 0; i < zeroes; i++) {
            if(databaseStorage.get(i) == 1) {
                swapForZeroesFront++;
            }
        }

        return Math.min(swapForZeroesFront, swapForOnesFront);

    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
```

```java

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0509 sec	37.3 KB
TestCase 1	Easy	Sample	Success	1	0.0681 sec	37.5 KB
TestCase 2	Easy	Sample	Success	1	0.057 sec	37.6 KB
TestCase 3	Easy	Hidden	Success	3	0.0534 sec	37.6 KB

TestCase 4	Easy	Hidden	Success	3	0.0678 sec	37.5 KB
TestCase 5	Easy	Hidden	Success	3	0.0545 sec	37.1 KB
TestCase 6	Easy	Hidden	Success	4	0.0492 sec	37.6 KB
TestCase 7	Easy	Hidden	Success	4	0.0503 sec	37.5 KB
TestCase 8	Easy	Hidden	Success	5	0.0539 sec	37.1 KB
TestCase 9	Easy	Hidden	Success	5	0.0482 sec	37.5 KB
TestCase 10	Easy	Hidden	Success	5	0.0621 sec	37.6 KB
TestCase 11	Medium	Hidden	Success	7	0.0643 sec	41 KB
TestCase 12	Medium	Hidden	Success	8	0.0749 sec	39.5 KB
TestCase 13	Hard	Hidden	Success	10	0.0817 sec	39.6 KB
TestCase 14	Hard	Hidden	Success	15	0.2839 sec	69.1 KB

No comments.